

Exercício 6.1 - Semáforos - Sincronização e Exclusão mutua

Implemente os seguintes programas seguindo o pseudocódigo estilo tipo-C usado por William Stallings (autor do livro da bibliografia principal).

- a) Utilize um semáforo para impor a ordem entre duas threads. A Thread A deve imprimir (print) "Hello" antes da Thread B imprimir "World!". Considere que cada Thread executa uma função distinta *void A()* {... *print("Hello")* ...} e *void B()* {... *print("World!")* ...}.
- b) Implemente uma secção crítica onde várias threads incrementam um contador partilhado. Utilize um semáforo para garantir que apenas uma thread modifica o contador de cada vez. Considere que cada thread executa uma função *void increment()* e modifica a variável *int count*.
- c) Adicione à solução da alínea anterior instruções para imprimir o valor de *count* que tinha quando foi incrementado. Considere que a thread pode executar outras instruções após incrementar e imprime o valor antes de terminar. A função *print* é uma secção crítica? E se imprimir o valor atual de *count* (podendo ser alterado por outras threads) antes da thread terminar?

Exercício 6.2 - Semáforos - cliente-servidor

Considere o problema de um cliente e um servidor (poderia ser um cliente de um restaurante e um empregado de um restaurante....)

- existem clientes e servidores (representados pelas funções *cliente()* e *servidor()*) que querem efetuar uma transação, no exemplo de num restaurante, poderia ser almoçar... essa ação é representada numa função *almoçar()*.

- o cliente só pode executar a função *almoçar()* se houver um servidor preparado para servir, ou seja pronto a avançar e a executar a função *atender()*;

- o servidor está durante um tempo indeterminado (mas finito) na função *atender()* e depois pode sair, e regressa mais tarde (considere um ciclo infinito) à função *atender()*;

- o servidor atende um cliente de cada vez

Implemente o programa seguindo o pseudocódigo estilo tipo-C usado por William Stallings (autor do livro da bibliografia principal).

Exercício 6.3 - Semáforos - leitores-escritores

1) Considere o problema clássico leitores-escritores (Readers/Writers) em que:

- os leitores para lerem chamam uma função *ler()*
- os escritores chamam a função *escrever()*
- quando temos um escritor mais nenhum processo pode aceder (mais nenhum processo pode chamar *ler()* ou *escrever()*)
- quando um leitor está a *ler()* outros leitores também podem chamar a função *ler()* mas não os escritores.

Pressupostos gerais:

a) usaremos um estilo de programação seguindo o estilo "tipo C" usado na bibliografia (W. Stallings) e pretende-se definir funções *leitor()* e *escritor()* que representam processos concorrentes e serão inicializados num *main()*, por exemplo:

```
void main()
{
    etc...
    parbegin (leitor, escritor);
}
```

b) cada processo concorrente é representado por uma função que executa em ciclo indefinidamente, por exemplo com um ciclo "while(true)":

```
void escritor()
{
    while (true) {
        semWait(x);
        escrever();
        etc...
        semSignal();
    }
}
```

c) no início deve ser feita a declaração e inicialização dos semáforos usados, e.g.:

semaphore x = 1; deste modo o resultado final deve ser algo como:

```
# Isto é apenas um exemplo, devem alterar os valores iniciais
```

```
semaphore x = 1, y = 0, z = 2;
```

```

int e;

...

void leitor()

{

    while(true) {

        etc ...

        semWait(x);

        etc...

    }

}

void escritor()

{

    while (true) {

        etc ...

    }

}

void main(){

e = 0;

...

parbegin (leitor, escritor);

}

```

2) Para cada um dos semáforos usados, descreva brevemente a sua função.

3) Existe risco de starvation para algum dos processos? Assuma que vários leitores e escritores podem estar a executar em “simultâneo”.

Exercício 6.4 - Semáforos - smokers/non-smokers

1) Considere o problema de acesso a um espaço (um restaurante, por exemplo) por clientes de dois tipos "fumadores" e "não-fumadores" representados por dois tipos de processos fumador e não-fumador, com as seguintes regras de acesso:

- cada processo pretende aceder ao restaurante e quando tem sucesso executa a função `estar()`;
- se existirem fumadores no restaurante o restaurante é só para fumadores;
- se existirem não-fumadores no restaurante o restaurante é só para não-fumadores;
- quando não há clientes no restaurante, poderá entrar um cliente de qualquer um dos tipos;

Proponha a solução mais simples em que se admite a possibilidade de haver starvation dos processos. Implemente o programa seguindo o estilo tipo-C usado por William Stallings.

2) (Extra) Crie uma nova implementação para o mesmo problema, mas em que garantidamente nenhum tipo de cliente pode ficar em starvation.